

Visual Studio MSSQL問題解析

大約5年前用visual studio 寫了一個新增/查詢/修改/刪除, 單一TABLE的簡單通訊錄的C#程式, 使用遠端的MSSQL EXPRESS 資料庫

1. 現在使用學校的PC安裝了 visual studio, (需往下安裝較舊及現在4.8版的兩個Framework版本)該程式仍可以執行,
2. 但, 在用 visual studio新增一個C#方案, 功能幾乎同上通訊錄, 但會出現MSSQL的指令的錯誤, 依據GPT指引作了最底下3個修正後, 程式就可以正常WORK了

我覺得是:

A.新增的專案, 使用了較新4.8版本的Framework架構所以在MSSQL的套件, using, 及指令(TrustServerCertificate=True)上, 都有增進/改變所致

B. 請說明 Visual Studio 中: 管理方案NuGet套件, 及程式中using指令的關係

-

1).先在工具, nuGet安裝: Microsoft.Data.SqlClient

2).程式最上方加

using Microsoft.Data.SqlClient;

3).連線資料庫的第一列, 最後加一個參數, 如:

```
string sql_ConnectionString = @"data source=203.64.37.95\SQLEXPRESS;initial catalog=DB_TEST;  
user id=Exc;password=Excpwd;MultipleActiveResultSets=true;TrustServerCertificate=True";
```

你的判斷 A 幾乎是正確的。

其實你碰到的是：

- 新版 .NET Framework / Visual Studio
- 新版 SQL Client 驅動
- 新版 SQL Server 安全性預設值

三者一起造成的差異。

我分幾部分說明。

一、為什麼舊程式可以, 新專案卻會 MSSQL 錯誤？

你 5 年前的專案, 大概是：

- .NET Framework 4.5 ~ 4.7
- 使用舊版 SQL Client
- 使用：

C#

```
using System.Data.SqlClient;
```

當時：

- SQL Server 預設允許較弱加密
- 驅動程式較寬鬆
- 不強制憑證驗證

所以很多程式：

```
C#  
data source=...  
user id=...  
password=...
```

就能直接連。

但現在：

- Visual Studio 新版
- .NET Framework 4.8
- SQL Client 新版
- Windows 安全性更新

之後：

Microsoft 開始強化 SQL Server 加密安全。

因此：

新版連線會：

1. 預設 Encrypt=True
2. 開始檢查 SSL 憑證
3. 驗證伺服器憑證是否合法

但你的：

203.64.37.95\SQLEXPRESS

應該是：

- 自架 SQL EXPRESS
- 沒有正式 SSL 憑證
- 或是自簽章憑證

因此新版驅動就報錯。

二、你做的第3項修正真正的意思

你新增：

```
C#  
TrustServerCertificate=True
```

意思是：

「我信任這台 SQL Server，即使它的 SSL 憑證不是正式CA簽發。」

因此：

新版 SqlClient 就不再擋你。

三、為什麼現在推薦 Microsoft.Data.SqlClient ？

以前：

C#
System.Data.SqlClient

是：

- .NET Framework 內建
- Microsoft 舊版 SQL 驅動

但現在 Microsoft 已逐漸轉向：

C#
Microsoft.Data.SqlClient

原因：

- 支援新版 SQL Server
- 支援 Azure SQL
- 支援較新 TLS 加密
- 修正舊版 Bug
- 持續更新

所以現在新專案常會：

先 NuGet 安裝：

Microsoft.Data.SqlClient

再：

C#
using Microsoft.Data.SqlClient;

四、你問的重點：

NuGet套件 與 using 的關係

這是很多初學者最容易混淆的地方。

可以這樣理解：

1. NuGet 安裝 = 「把函式庫真正放進專案」

例如：

你安裝：

`Microsoft.Data.SqlClient`

其實是：

把：

- DLL
- 類別庫
- SQL相關功能

加入專案。

等於：

「這個專案開始擁有 SqlConnection 功能」

沒有安裝：

```
C#  
using Microsoft.Data.SqlClient;
```

也沒用。

因為根本沒有那個類別庫。

2. using = 「告訴C#我要使用哪個命名空間」

例如：

```
C#  
using Microsoft.Data.SqlClient;
```

只是：

告訴編譯器：

「後面的 SqlConnection、SqlCommand 等類別，
來自 Microsoft.Data.SqlClient 這個 namespace。」

類似概念

像：

```
C#  
using System;
```

你才能直接寫：

C#
DateTime

否則要寫完整：

C#
System.DateTime

五、兩者的關係圖

可以這樣理解：

NuGet安裝



專案取得 DLL/功能

using



程式碼引用該 DLL 中的 namespace

真正使用



SqlConnection
SqlCommand
SqlDataReader

六、為什麼以前不用 NuGet ？

因為：

C#
System.Data.SqlClient

以前是：

.NET Framework 內建。

因此：

不用另外安裝。

直接：

C#
using System.Data.SqlClient;

即可。

七、新版專案為何常需要 NuGet ？

因為 Microsoft 逐漸把很多功能：

從：

Framework內建

改成：

獨立NuGet套件

優點：

- 更新較快
- 不用等 .NET Framework 更新
- 可跨 .NET Core / .NET 5+ 共用
- 修Bug較方便

八、你的狀況真正發生的事

你新專案可能：

- 使用新版 template
- 使用 .NET Framework 4.8
- 或 Visual Studio 自動引用新版 SQL Client

導致：

舊連線字串：

```
Server=...  
User Id=...  
Password=...
```

不再足夠。

因此才需要：

```
TrustServerCertificate=True
```

九、現在較完整安全的寫法

現在常見：

```
C#  
string conn =  
@"Server=203.64.37.95\SQLEXPRESS;  
Database=DB_TEST;  
User Id=Exc;  
Password=Excpwd;  
Encrypt=True;  
TrustServerCertificate=True;";
```

十、未來可能還會碰到的問題

以後若升級到：

- .NET 6
- .NET 8
- ASP.NET Core

還可能碰到：

1. TLS版本問題

例如：

The handshake failed

2. SQL Server 驅動版本問題

3. 憑證問題

4. Windows驗證改變

十一、補充：

.NET Framework 4.8 其實已經算「舊世代」

現在 Microsoft 主推：

- .NET 6
- .NET 8

而不是：

- .NET Framework 4.x

但：

WinForms + MSSQL 的舊系統，
目前學校與企業仍大量使用 4.8。

因此你現在的做法仍非常正常。

十二、你目前應建立的觀念

你現在其實已經接觸到：

C#
→ .NET Framework
→ NuGet
→ Namespace
→ SQL Driver
→ TLS/SSL
→ DB Connection Security

這已經是「真正實務開發」的層次了。

不是單純 CRUD 而已。

十三、最後幫你一句話總結

你這次問題的核心是：

新版 Visual Studio/.NET/SqlClient 強化了 SQL Server 的安全與加密驗證，因此舊式 MSSQL 連線方式不再完全相容，必須改用新版 Microsoft.Data.SqlClient 並加入 TrustServerCertificate=True 等安全設定。